



UNIVERSITÉ SAINT JEAN
SAINT JEAN INGÉNIEUR

Tél : (+237) 657 07 98 07

(+ 237) 651 36 96 96

B.P : 749 Yaoundé, Cameroun

Email : info@institutssaintjean.org

www.institutssaintjean.org

Intitule de l'UE : **Projet**

Code de l'UE : IS4177

Intitule de l'EC : Projet Transversal ISI, Techdays

Code de l'EC : **ING4167**

Groupe no 3

Gestion des Emplois de Temps et Réservations de salles à l'IUSJC :

Cahier de conception

Membres du groupe :

BELINGA Astrid

MELONG Joan

DONGMO Leslie

Sous la supervision de :

Dr MOUPOJOU Emmanuel

Année académique 2025-2026

1. Introduction.....	3
2. Présentation du projet.....	3
2.1. Objectifs du système.....	3
2.2. Interfaces du système.....	3
2.3. Contraintes générales de conception.....	3
3. Documents de référence.....	4
4. Normes, standards et outils.....	4
4.1. Méthodes de conception.....	4
4.2. Environnement et outils de développement.....	4
4.2.1. Matériels et Outils.....	4
4.2.2. Standard de Programmation.....	4
4.3. Plan de Qualité.....	5
5. Conception générale.....	5
5.1. Identification et description des modules.....	5
5.2. Diagramme de composants.....	5
5.3. Diagramme de déploiement.....	6
6. Conception détaillée.....	7
6.1. Diagramme de classes de conception.....	7
6.2. Algorithmes.....	9
6.3. Thème de l'application.....	9
6.4. Maquettes des principales interfaces.....	9
7. Conclusion.....	9

1. Introduction

Ce document constitue le Cahier de Conception de l'application de gestion des emplois du temps et des réservations de salles de l'Institut Universitaire Saint-Jean (IUSJC). Il s'inscrit dans la continuité du Cahier des Charges Technique et du Cahier d'Analyse précédemment réalisés. L'objectif de ce document est de préciser la structure logicielle du système, son architecture d'exécution, ses modules fonctionnels et techniques, ainsi que les algorithmes et normes qui guideront son implémentation. Il sert de base de référence à l'équipe de développement pour la phase de codage et d'intégration.

2. Présentation du projet

2.1. Objectifs du système

L'application vise à automatiser la gestion des emplois du temps et des réservations de salles au sein de l'IUSJC. Les principaux objectifs sont :

- Planifier automatiquement les cours selon la disponibilité des enseignants, des salles et des classes.
- Réduire les conflits d'occupation et proposer des alternatives de créneaux ou de salles.
- Permettre aux enseignants et aux étudiants de consulter leurs emplois du temps et cela en temps réel.
- Générer des rapports statistiques (taux d'occupation, disponibilités, etc.).
- Synchroniser les calendriers avec des outils externes (Google Calendar, Outlook, ENT).

2.2. Interfaces du système

Le système interagit avec plusieurs interfaces externes, hors interfaces graphiques :

- **ENT/Annuaire IUSJC** : pour l'import/export des utilisateurs et des données de calendrier (via API REST ou iCal).
- **Google / Outlook Calendar** : pour la synchronisation bidirectionnelle des événements.
- **SMTP / API SMS (Orange/MTN)** : pour l'envoi de notifications (emails, SMS).

2.3. Contraintes générales de conception

- **Sécurité** : utilisation de protocoles HTTPS/TLS, authentification JWT et gestion des rôles.
- **Performance** : temps de réponse inférieur à 2 secondes pour les opérations courantes.
- **Disponibilité** : sauvegardes planifiées et tolérance aux pannes via conteneurisation.
- **Interopérabilité** : architecture RESTful, compatibilité avec les API externes.
- **Accessibilité** : application bilingue (FR/EN) et interface responsive.
- **Protection des données** : conformité au RGPD et à la législation camerounaise.

3. Documents de référence

Les documents suivants ont servi de base à l'élaboration du présent Cahier de Conception :

- Cahier des charges technique
- Cahier d'analyse
- Notes et livrables des phases précédentes (analyse et conception préliminaire).
- Référentiels de normes ISO 12207, IEEE 730, IEEE 1028, IEEE 1059, ISO 27005.

Norme	Sujet	Accès gratuit
ISO 12207	Cycle de vie logiciel	iso.org/obp/ui
IEEE 730	Plan d'assurance qualité logicielle	Résumé sur IEEE Xplore / https://standards.ieee.org/
IEEE 1028	Revue et audit logiciel	Résumé sur IEEE Xplore / https://standards.ieee.org/
IEEE 1059	Vérification et validation	Résumé sur IEEE Xplore / https://standards.ieee.org/
ISO 27005	Gestion des risques sécurité	iso.org/obp/ui

4. Normes, standards et outils

4.1. Méthodes de conception

La conception du système s'appuie sur une démarche agile de type **SCRUM**, permettant une livraison incrémentale et itérative des fonctionnalités. Le langage de modélisation retenu est **UML** (Unified Modeling Language), en raison de sa capacité à représenter clairement les aspects structurels et comportementaux du système.

4.2. Environnement et outils de développement

4.2.1. Matériels et Outils

- **Langages** : Java 17 (backend), HTML/CSS/JS (frontend).
- **Frameworks** : Spring Boot / Spring Cloud, Thymeleaf, Docker, PostgreSQL, Kafka.
- **Outils de collaboration** : GitHub, Trello, Swagger, SonarQube.
- **Tests et supervision** : JUnit, Prometheus, Grafana, ELK Stack.

4.2.2. Standard de Programmation

- Respect des conventions de nommage Java et de la PEP8 pour le Python.
- Documentation des classes et méthodes (JavaDoc).
- Tests unitaires systématiques (>80% de couverture).
- Utilisation de branches Git (feature / develop / main) et validation par merge requests.

4.3. Plan de Qualité

Le plan de qualité est conforme aux standards suivants :

- Application de l'**Assurance Qualité Logicielle** (AQL) selon la norme **IEEE 730**.
- **Vérification et validation** (V&V) selon la norme **IEEE 1059**.
- **Revue et Audits** définis par la norme **IEEE 1028**.
- **Gestion de configuration** via **GitHub**.
- **Gestion des risques** selon la norme **ISO 27005**.

5. Conception générale

5.1. Identification et description des modules

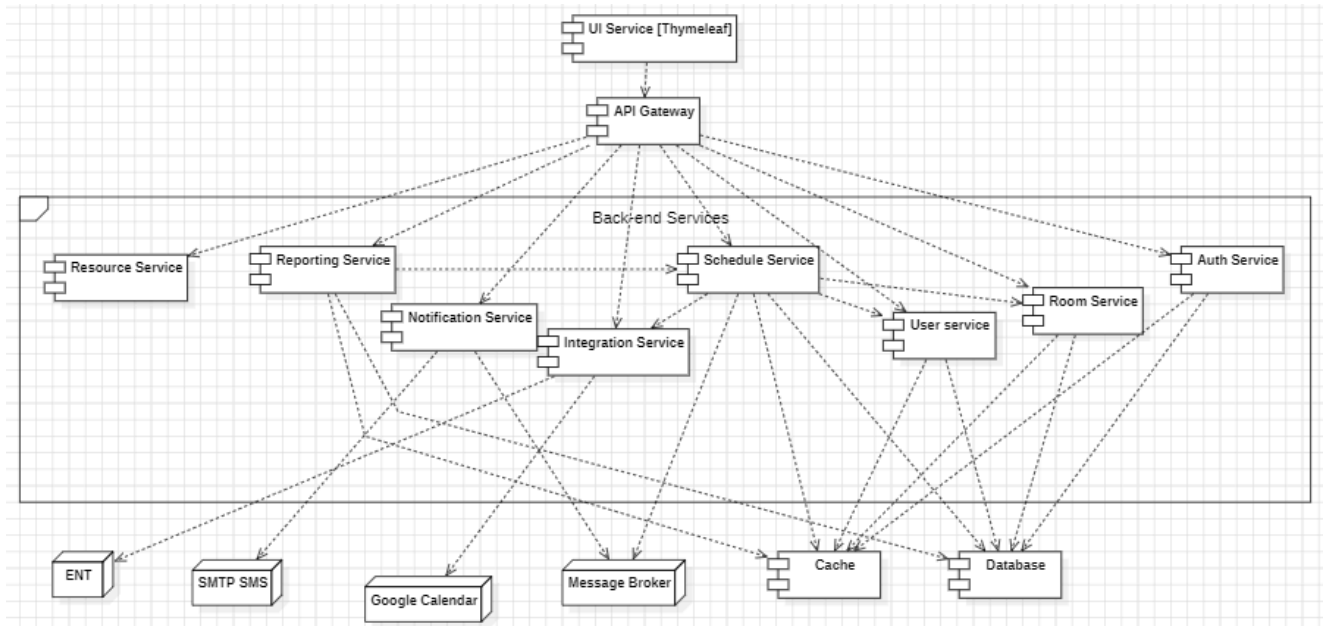
Les principaux modules logiciels du système sont :

- **Auth Service** gère l'authentification et les rôles des utilisateurs.
- **User Service** : assure la gestion des profils (administrateurs, enseignants, étudiants).
- **Schedule Service** : planifie les cours, détecte les conflits et génère les emplois du temps.
- **Room Service** : gère la disponibilité et la réservation des salles.
- **Resource Service** : gère les équipements matériels.
- **Notification Service** : envoie les emails et SMS.
- **Reporting Service** : produit les rapports statistiques.
- **Integration Service** : synchronise avec l'ENT et les calendriers externes.
- **UI Service (Thymeleaf)** : interface utilisateur web.
- **API Gateway et Service Discovery (Eureka)** : orchestration des services.



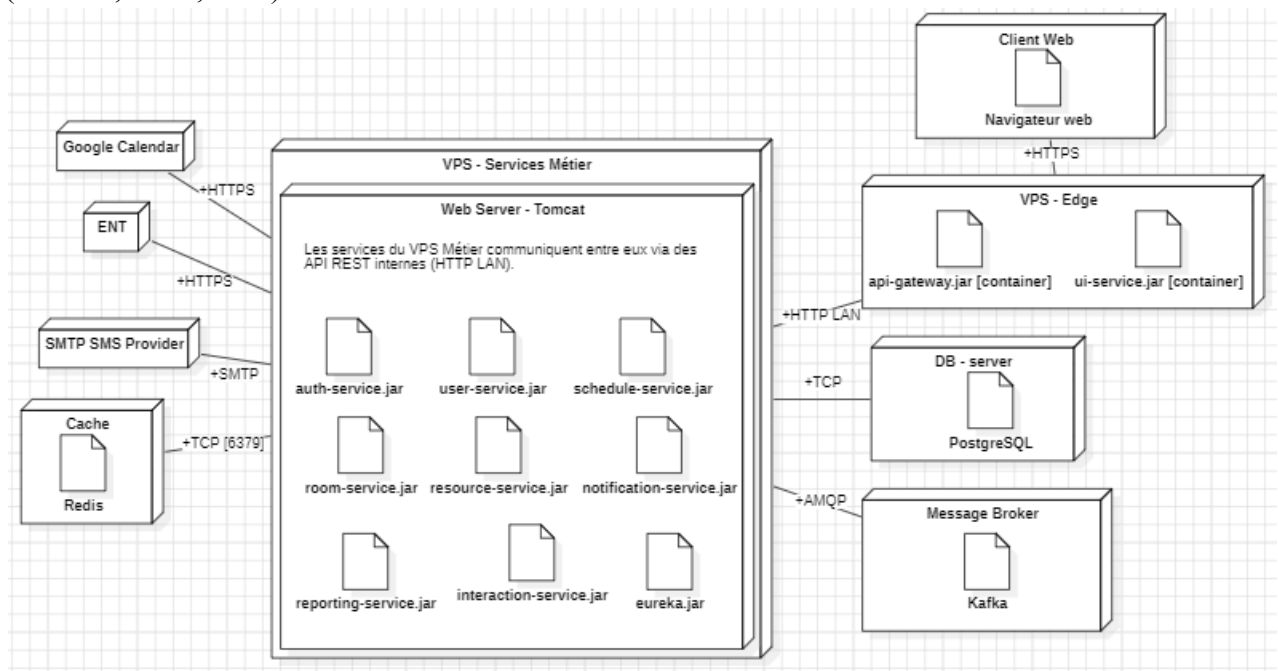
5.2. Diagramme de composants

Le diagramme de composants décrit la structure modulaire du système et les relations entre les services. Chaque composant représente un module indépendant, encapsulant ses données et ses interfaces. Les services communiquent principalement via des API REST sécurisées.



5.3. Diagramme de déploiement

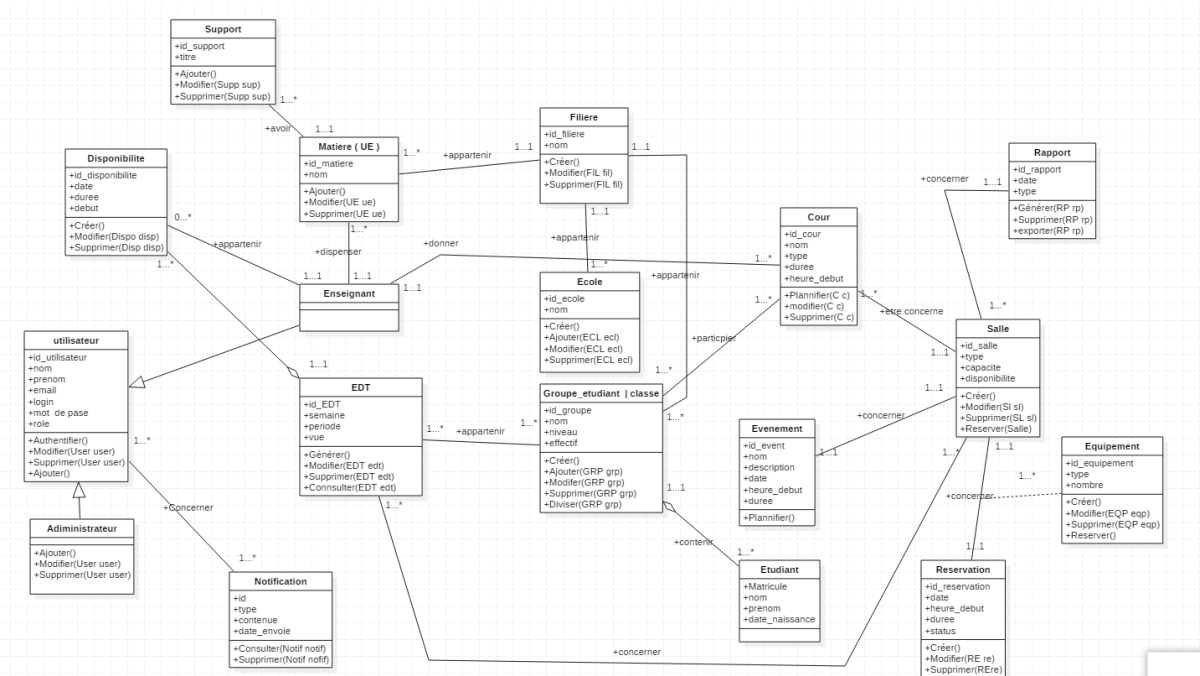
Le diagramme de déploiement montre la répartition des composants logiciels sur les nœuds matériels (serveurs, base de données, client web, etc.). Il décrit les connexions réseau (Internet, LAN, TCP).



6. Conception détaillée

6.1. Diagramme de classes de conception

Le diagramme de classes de conception reprend les entités principales identifiées dans la phase d'analyse, avec leurs attributs typés et les signatures de méthodes. Les classes clés incluent : École, Filière, Classe, Étudiant, Groupe, Enseignant, Matière, Salle, Cours, EmploiDuTemps, Notification, Rapport, etc. Les classes Étudiant et Classe ont été ajoutées pour modéliser la hiérarchie pédagogique, et la classe Groupe permet de gérer les options et TD spécifiques.



- Groupes d'options/TD spécifiques modélisés par l'entité Groupe et la table d'adhésion GroupeAdhesion.
- Un Cours cible soit une Classe entière soit un Groupe (colonne groupe_id nullable si classe_id présent).
- Respect de la hiérarchie Matière → Classe → Filière → École.

6.2. Algorithmes

Les principaux algorithmes du système sont :

- **Détection de conflits** : vérifie les chevauchements de créneaux pour les enseignants, salles et classes.
- **Attribution automatique de salle** : sélectionne la salle optimale selon la capacité et la disponibilité.
- **Optimisation hebdomadaire** : minimise les conflits et équilibre les charges par heuristique gloutonne ou recuit simulé.

6.2.1. Algorithme de détection de conflits

Objectif : Identifier si deux cours (ou plus) se chevauchent pour une même **ressource** : enseignant, salle, ou classe/groupe.

Idée : On peut utiliser un **algorithme de balayage temporel** (*sweep line algorithm*), similaire à ceux utilisés en planification de tâches. On trie tous les créneaux par heure de début, puis on parcourt en vérifiant les chevauchements avec une structure de file active.

Pseudo-code

ALGORITHME Détection_Conflits(listeCours)

Entrée : liste des cours avec attributs (début, fin, enseignant, salle, classe)

Sortie : liste des conflits détectés

conflits ← []

POUR chaque typeRessource DANS {enseignant, salle, classe} FAIRE

Grouper les cours par ressource

groupes ← regrouperPar(listeCours, typeRessource)

POUR chaque ressource DANS groupes FAIRE

coursRess ← trier(groupes[ressource], par=début)

POUR i DE 0 À taille(coursRess)-2 FAIRE

c1 ← coursRess[i]

c2 ← coursRess[i+1]

SI c2.début < c1.fin ALORS

ajouter(conflits, (typeRessource, ressource, c1, c2))

FIN SI

FIN POUR

FIN POUR

FIN POUR

RETOURNER conflits

FIN

Complexité

- Tri : $O(n \cdot \log(n))$ par ressource.
- Balayage : $O(n)$.
- Total $\approx O(n \log n)$ — très efficace même pour de gros volumes.

Utilisation :

Quand un enseignant, une classe, ou une salle tente d'ajouter un cours,
→ le service **ScheduleService** exécute cet algorithme.
→ si conflit, on renvoie une suggestion de créneau libre ou de salle alternative.

6.2.2. Algorithme d'attribution automatique de salle

Objectif : Attribuer à chaque cours une **salle disponible** compatible avec :

- le **type de cours** (TD, TP, CM, etc.),
- la **capacité minimale** ($\geq$ effectif classe/groupe),
- les **contraintes temporelles** (pas de conflit).

Idée : C'est un problème d'**affectation pondérée** (cours $\rightarrow$ salles) : On calcule un **score** pour chaque salle et on choisit la meilleure disponible.

Pseudo-code

ALGORITHME Attribution_Salle(listeCours, listeSalles)

Entrée : liste des cours sans salle, liste des salles existantes

Sortie : association cours $\rightarrow$ salle optimale

associations $\leftarrow []$

POUR chaque cours DANS listeCours FAIRE

salleCandidats $\leftarrow []$

POUR chaque salle DANS listeSalles FAIRE

SI salle.type = cours.typeCours ET salle.capacite $\geq$ cours.effectif ALORS

Vérifier disponibilité de la salle au créneau du cours

SI NON existeConflit(salle, cours.debut, cours.fin) ALORS

score $\leftarrow$ (salle.capacite - cours.effectif) + bonusProximite(salle, cours.classe)

ajouter(salleCandidats, (salle, score))

FIN SI

FIN SI

FIN POUR

Sélection de la salle avec score minimal

salleChoisie $\leftarrow$ min(salleCandidats, par=score)

associer(cours, salleChoisie)

ajouter(associations, (cours, salleChoisie))

FIN POUR

RETOURNER associations

FIN

Complexité

O(n × m) où n = nbCours et m = nbSalles

→ acceptable pour des centaines de cours ($< 10^4$ comparaisons).

Utilisation

Quand un administrateur planifie un nouveau cours :

→ **ScheduleService** exécuté Attribution_Salle()

→ la salle sélectionnée est enregistrée en base avec le cours.

6.2.3. Algorithme d'optimisation hebdomadaire

Objectif : Sur une semaine entière :

- minimiser les **chevauchements**,
- équilibrer la **charge horaire** des enseignants et classes,

- maximiser le **taux d'occupation des salles**.

Idée : C'est un **problème de flot maximum ou d'affectation binaire**. Chaque créneau (cours potentiel) est un **nœud source**, chaque ressource (salle, enseignant, classe) est un **nœud de destination**.

On peut modéliser :

- les **créneaux** comme les arêtes du graphe,
- les **contraintes de disponibilité** comme des **capacités**,
- puis appliquer **Ford–Fulkerson** pour maximiser le nombre d'affectations valides (cours planifiés).

Étape 1 — Modélisation en graphe

- **Source (S)** → ensemble de **Cours à placer**
- **Salles, Enseignants, Classes** → nœuds intermédiaires
- **Sink (T)** → fin du flot (cours planifiés avec ressources compatibles)

Chaque arête a une **capacité** :

- 1 si la salle/enseignant est disponible, 0 sinon.
- Poids = coût (distance, préférence, contrainte douce).

Étape 2 — Algorithme Ford–Fulkerson

ALGORITHME Optimisation_Hebdomadaire_FF(Graphe G, Source S, Sink T)

flotMax $\leftarrow$ 0

TANT QU'il existe un chemin augmentant P de S à T dans G (via BFS/DFS)

capacitéRésiduelle $\leftarrow$ min(capacité(e) pour e dans P)

POUR chaque arête e dans P FAIRE

diminuer capacité(e) de capacitéRésiduelle

augmenter capacité inverse(e) de capacitéRésiduelle

FIN POUR

flotMax $\leftarrow$ flotMax + capacitéRésiduelle

FIN TANT QUE

RETOURNER flotMax

FIN

Étape 3 — Application concrète

1. Construis le graphe :
 - chaque cours non encore placé → arête depuis Source.
 - chaque ressource libre → arête vers Sink.
 - lien entre cours et ressource s'il est possible (heure libre, type compatible).
2. Applique Ford–Fulkerson pour trouver le **maximum de cours compatibles planifiés** sans conflit.
3. Les arêtes du flot maximal indiquent l'affectation optimale.

Complexité : Si **E** est le nombre d'arêtes et **V** le nombre de sommets :

O(E × F) où **F** = flot maximal (nombre d'affectations).

→ efficace sur un graphe de taille moyenne (quelques centaines de cours/ressources).

Étape 4 — Amélioration heuristique (recuit simulé / glouton)

Une fois les affectations de base trouvées :

- on applique une **heuristique gloutonne** pour corriger les déséquilibres (ex. trop de cours consécutifs).
- ou un **recuit simulé** :
 - perturbe légèrement une solution,
 - accepte des changements sous probabilité $p = \exp(-\Delta E / T)$,
 - réduit progressivement **T** (température).
 - optimise des critères : **chargeEnseignant**, **occupationSalle**, **tempsMoyCours**.

Utilisation :

- Une fois par semaine (ou sur demande admin), **ScheduleService** exécute :
Optimisation_Hebdomadaire_FF() → pour maximiser les affectations viables,
puis **Heuristique_Gloutonne()** / **Recuit_Simule()** → pour équilibrer les charges.

Récapitulatif synthétique

Fonction	Type d'algorithme	Objectif principal	Complexité	Module concerné
Détection de conflits	Balayage temporel	Identifier les chevauchements	$O(n \log n)$	ScheduleService
Attribution automatique de salle	Recherche multicritère gloutonne	Trouver salle optimale	$O(n \times m)$	RoomService + ScheduleService
Optimisation hebdomadaire	Graphe (Ford–Fulkerson) + heuristique	Maximiser le placement et équilibrer la charge	$O(E \times F)$	ScheduleService

6.3. Thème de l'application

- Logo officiel de l'application : IUSJ Planner



- Palette de couleurs

Primaire : #4F39F6

Secondaire : #000000 (40%)

Sombre : #101828

- Typographie : Inter (Titre en bold 34, sous-titre en bold 24, normal en 14)

6.4. Maquettes des principales interfaces

Les maquettes ont été réalisées sur Figma et sont disponibles depuis le lien suivant :
[figma-tutore-isi4-fr-grp3](#)

7. Conclusion

Le présent Cahier de Conception définit l'architecture logicielle et matérielle de l'application de gestion des emplois du temps et des réservations de salles. Il s'appuie sur une conception modulaire et évolutive garantissant la maintenabilité et la sécurité du système. La prochaine étape consistera à implémenter les modules décrits, à réaliser les tests unitaires et d'intégration, puis à procéder au déploiement progressif sur l'environnement de production.